

POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

Information System Analysis and Design

MULTIMODAL SEARCH SYSTEM

FOR E-COMMERCE

Assignment 06 – UML Design and Python Prototype

Student: Vu Hung Anh

Student ID: B23DCDT022

Submission date: 1 October 2026

Submission version: V01

Abstract. This report designs and demonstrates a small e-commerce search prototype for text, voice, and product images. The browser accepts live microphone speech or an editable transcript. The report includes the required UML diagrams, Python implementation, and tests on a teaching catalog and a separate real-photo sample.

1 Problem and requirements

Online customers search for products in different ways: they may type a description, speak a query, or provide a product image. The system converts each input to a common query format, retrieves candidate products, ranks them, and displays the results. Customers can also look up their own orders.

The prototype connects requirements analysis, UML design, a three-layer architecture, and executable Python code. It has a short console example and a local browser interface. The browser can recognize microphone speech and pass its transcript to Python; typed transcripts remain available. Image search uses Apple's Vision framework through a local Swift process on macOS.

Text, voice, or image → common query → candidate retrieval → ranking → results

Primary actor: Customer. A customer ID is supplied for order lookup; the repository checks ownership before returning an order.

Table 1: Functional requirements and implementation evidence

ID	Requirement	Evidence
FR01	Search products using text keywords or a short sentence.	Text normalization, filters, and ranked results.
FR02	Search by voice input.	Browser speech recognition creates a transcript; SpeechService sends it to text retrieval.
FR03	Search by product image.	ImageService validates the photo; Apple Vision encodes it for ranking.
FR04	Use one query format and separate retrieval from ranking.	QueryService, SearchService, and RankingService.
FR05	View a product and search or view a customer-owned order.	Product lookup and customer-scoped OrderService.
FR06	Demonstrate the input, processing, and result for each required mode.	Browser and console output, plus saved evaluation cases.

Table 2: Non-functional requirements

ID	Requirement and response
NFR01	Maintainability: the presentation calls services; repositories handle data access.
NFR02	Reproducibility: sample data, setup steps, and evaluation outputs are included.
NFR03	Explainability: ranked results show the component scores used by the prototype.
NFR04	Robustness and privacy: invalid empty inputs are rejected; order lookup checks customer ownership.

The system returns up to five ranked products with scores. A query with no eligible product returns no matches. An unknown order or an order belonging to another customer is not returned.

2 Use-case model

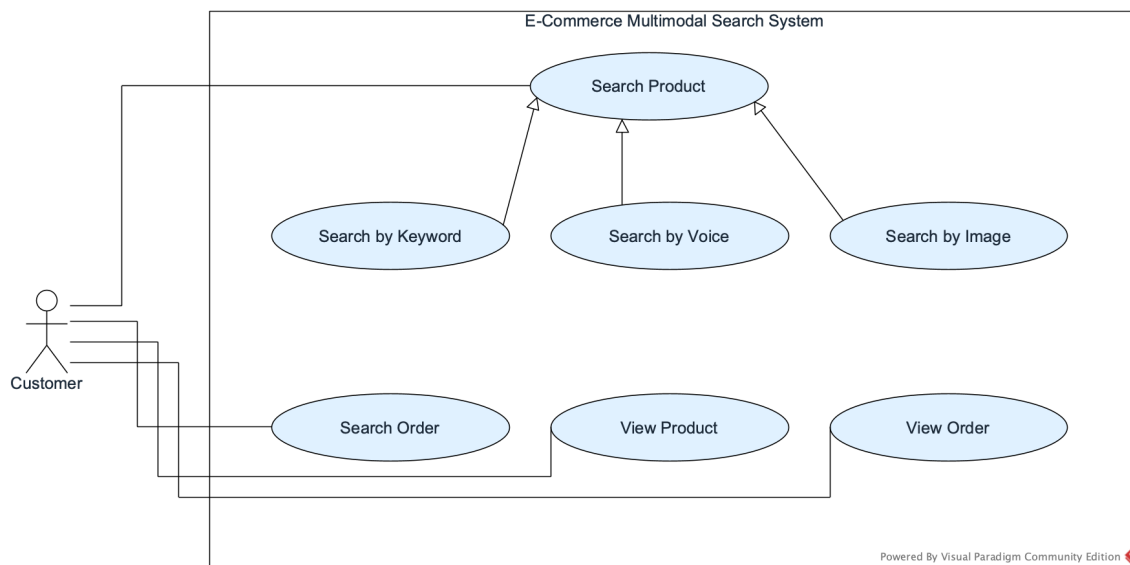


Figure 1: Use-case diagram created in Visual Paradigm.

The diagram has one Customer actor and all seven use cases required by the assignment. Search by Keyword, Search by Voice, and Search by Image specialize Search Product. Search Order, View Product, and View Order are separate customer goals. The actor is outside the system boundary; each use case is inside it.

Table 3: Main use-case flows

Use case	Main flow and result
Search Product	Customer submits text, a transcript, or an image. The system builds a query, retrieves and ranks products, then displays the top results.
Search Order	Customer supplies an order ID or asks for the latest order. The system checks ownership and returns the status, or reports no match.
View Product	Customer selects a product ID. The system returns its record or reports that it is unavailable.
View Order	Customer selects an order found under the same customer ID and sees its details.

3 Three-layer architecture

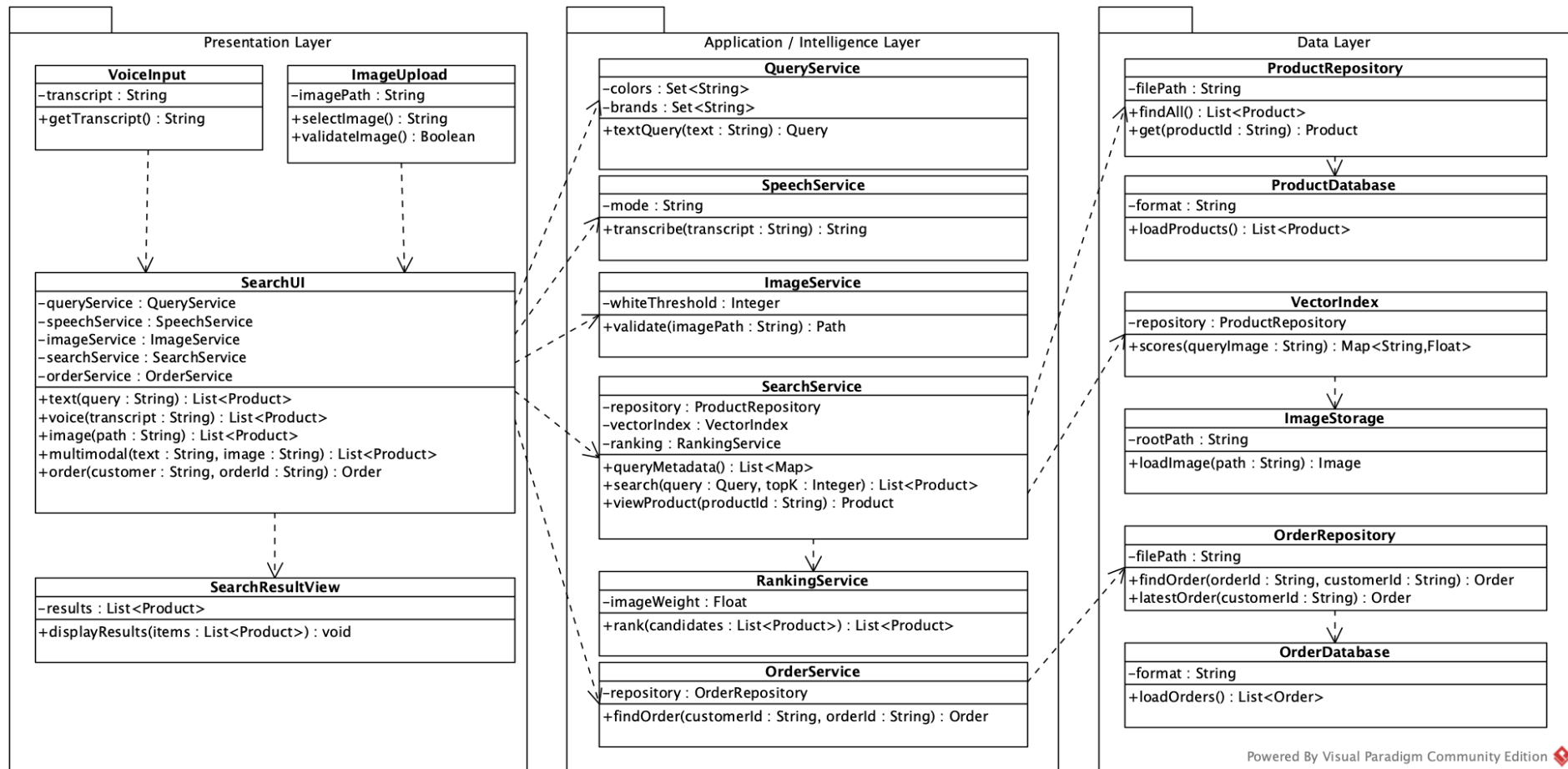


Figure 2: Three-layer class diagram with native Visual Paradigm classes and UML dependencies.

Presentation collects input and displays results. **Application / Intelligence** prepares queries, performs search and ranking, and handles orders. **Data** provides repositories, JSON records, product images, and the Vision image index. Dashed dependency arrows point from the class that uses a service to the class it uses. The main flow is SearchUI → application services → data services. Each class uses Visual Paradigm’s default name, attributes, and operations compartments.

4 Voice-search sequence

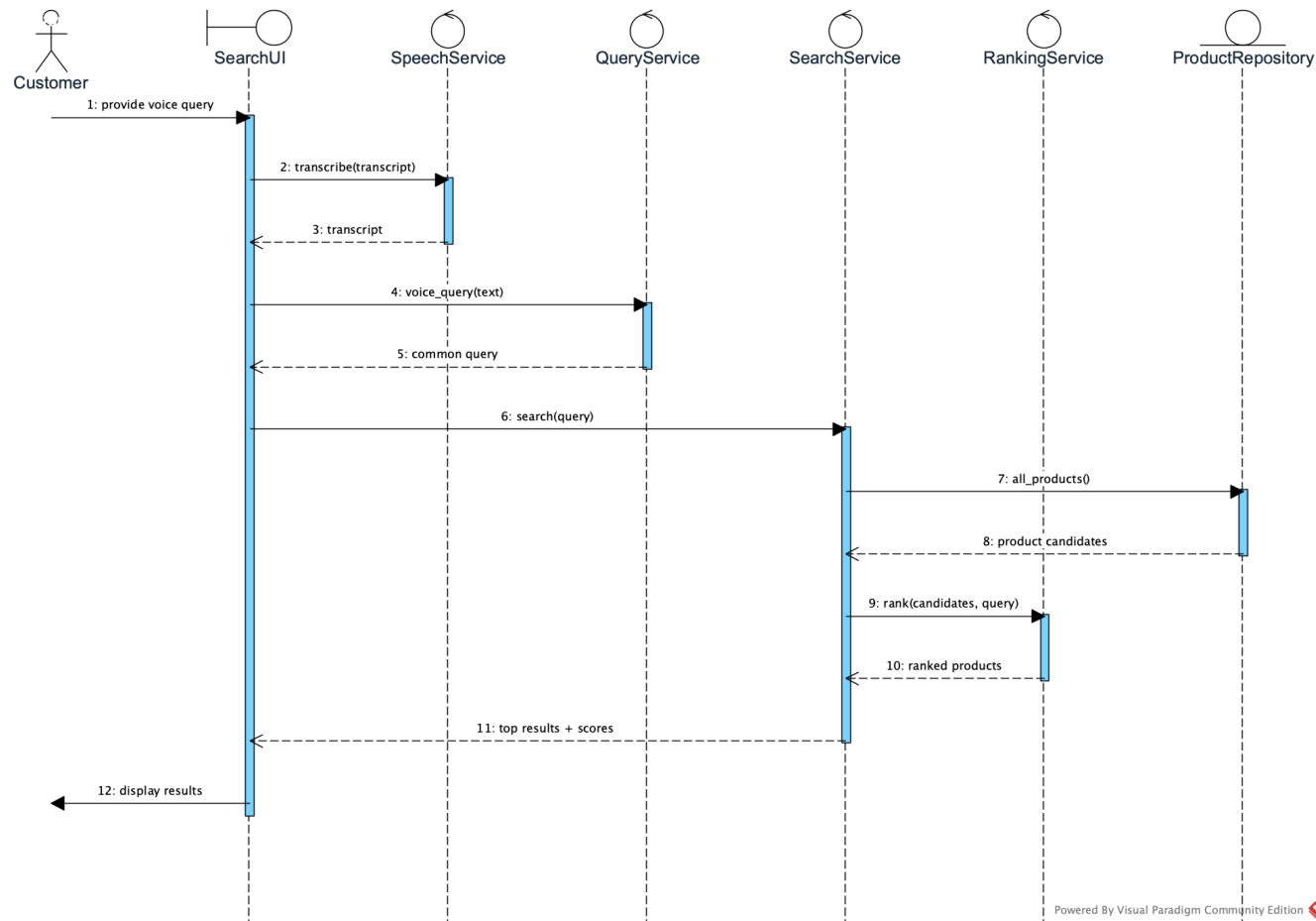


Figure 3: Voice-search sequence diagram. Boundary, control, and data-access roles are identified in the lifeline heads.

Main flow:

1. Customer provides voice input; browser speech recognition produces a transcript for SearchUI. The console version supplies that transcript directly.
2. SpeechService passes the text to QueryService, which creates a common query.
3. SearchService retrieves candidates from ProductRepository, ranks them, and returns results for SearchUI to display.

5 Data and implementation

The teaching catalog has 12 synthetic products and three sample orders. This is more than the assignment's ten-product minimum. It supports short text, voice-transcript, and order examples; its generated product pictures were removed. The browser's text, voice-transcript, image, and combined search instead use 500 real Open Food Facts products and photos.

Table 4: Data records used by the prototype

Record	Main fields	Storage
Product	ID, name, category, color, price, stock, image	data/products.json; real photos in data/openfoodfacts/images/
Order	ID, customer ID, date, status, total	data/orders.json
Order item	Product ID, quantity, unit price	Nested in an order record
Image feature print	Product photo and Vision image features	Indexed in memory by the local Swift process

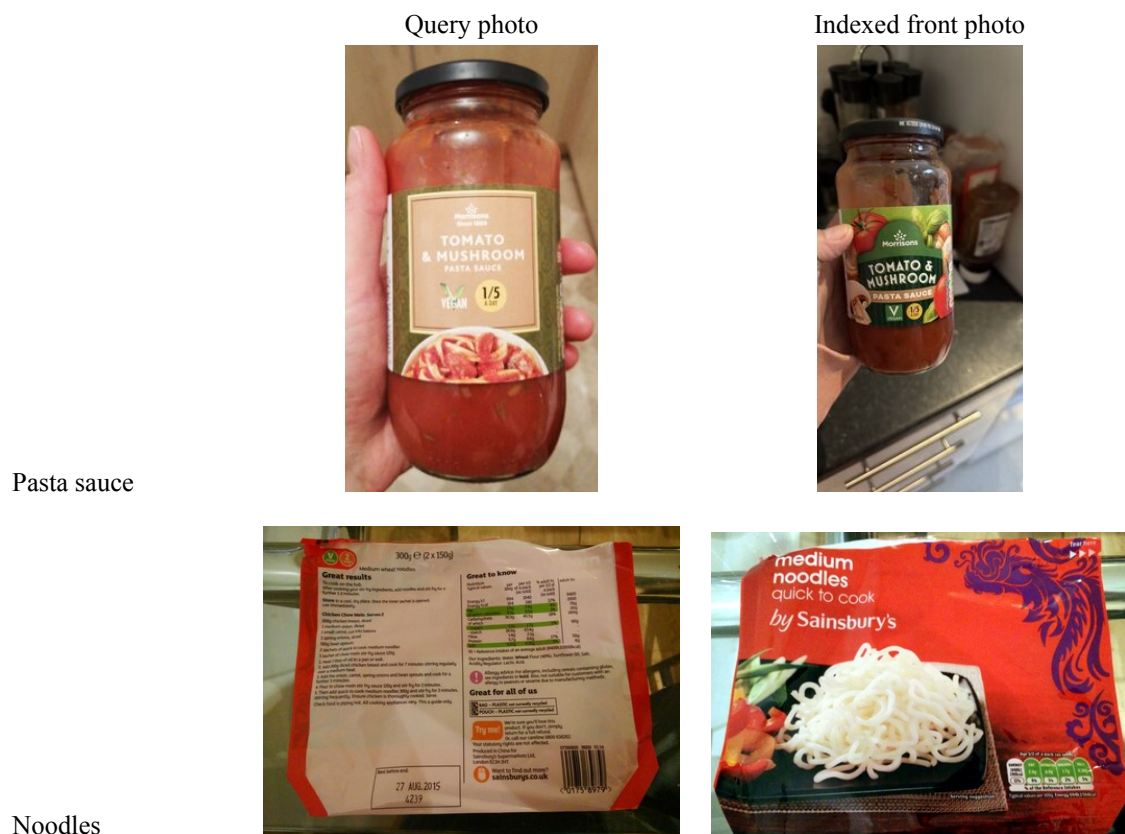


Figure 4: Real Open Food Facts query and gallery photos (CC BY-SA 3.0). The sauce is retrieved first; the noodles front photo is outside the top five.

Service responsibilities and search logic

Table 5: UML classes and Python modules

Layer / component	Implementation
Presentation	SearchUI collects mode-specific input and displays results.
Query handling	QueryService normalizes input into a common query.
Voice and image	SpeechService passes the transcript; ImageService validates the uploaded photo.
Product search	SearchService retrieves candidates; RankingService sorts them.
Order search	OrderService handles order ID and latest-order requests.
Data	Repositories load JSON; VectorIndex stores Vision feature prints for product photos.

Text and voice. QueryService lowercases text, normalizes a small set of category synonyms, and detects common colors, brands, and price limits. SearchService applies the detected filters before ranking. In the browser, the Web Speech API turns microphone speech into editable text. SearchUI then sends that transcript through SpeechService and the same query path. The console example and saved voice checks use supplied transcripts, so they do not measure recognition accuracy.

Image search. ImageService validates the uploaded file. VectorIndex uses Apple’s Vision feature-print API to describe each catalog photo and the query photo. Vision measures their visual distance d ; a smaller distance means a closer match. The application displays the equivalent similarity $S_i = 1/(1 + d)$, so a larger displayed score is better. The same Vision index is used by the running Python application and the evaluation script. It requires macOS and Swift.

Ranking. The business score B combines popularity and in-stock status. Text or voice search uses $0.9S_t + 0.1B$. Image-only search uses S_i , preserving Vision’s visual order. When both text and image are supplied, the score is $0.45S_t + 0.45S_i + 0.1B$. Retrieval selects eligible candidates; RankingService sorts those candidates by score.

External data. The browser and evaluation use 500 real grocery products and 100 separate alternate-view query images from Open Food Facts. Gallery and query photos are kept as distinct files; a query photo is never used in the gallery index. Every product has a barcode and title; 350 have a brand and 204 have a usable broad category. Source selection, attribution, and license details are in `data/openfoodfacts/DATASET.md`.

6 Demonstration

```
python main.py - captured output

=== E-Commerce Multimodal Search Demo ===
TEXT: Nike shoes under 100 dollars
type=text, input='find Nike shoes under 100 dollars', tokens=['nike', 'shoes'], category=shoes, color=None, brand=nike, max_price=100.0
Retrieval -> ranking -> top results:
1. Nike Air Zoom Black Running Shoes | score=0.9946 (text=1.000, image=0.000, business=0.946)
VOICE: black running shoes (transcript simulation)
type=voice, input='find black running shoes', tokens=['black', 'running', 'shoes'], category=shoes, color=black, brand=None, max_price=None
Retrieval -> ranking -> top results:
1. Nike Air Zoom Black Running Shoes | score=0.9946 (text=1.000, image=0.000, business=0.946)
```

Figure 5: Console text and simulated-voice demonstration generated by `python main.py`.

The console output shows the input, normalized query, ranked results, and scores for text and a supplied voice transcript. The order demo finds order 20261001 for customer C001; the same request returns no order for customer C002. In the browser, the Voice tab can capture speech from the microphone, show the recognized text, and run the same voice search.

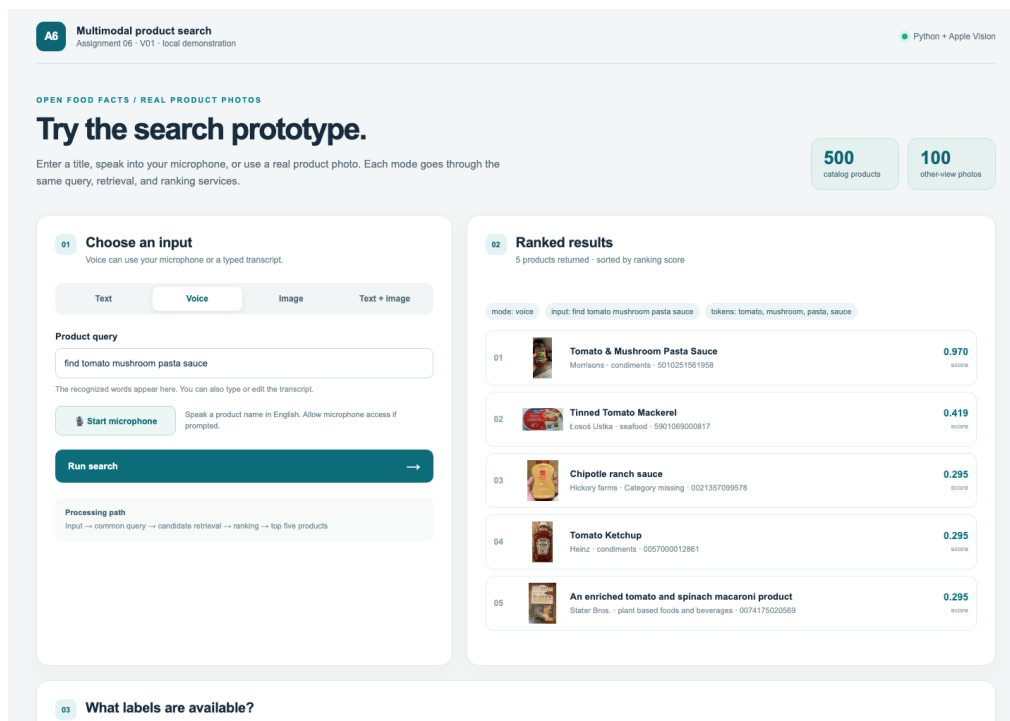


Figure 6: Browser Voice tab with microphone control and ranked results from an example transcript.

Table 6: Labels visible in the browser's real-product catalog

Field	Availability	How it is used
Barcode and name	500/500 products	Product identity and title lookup.
Brand	350/500 products	Display and keyword matching when present.
Broad category	204/500 products	Display; 52/100 image queries have a usable category check.
Query's true barcode	100/100 photos	Correct answer for exact-product image retrieval.

Shop prices and inventory are absent from Open Food Facts; the real-photo image ranking does not use placeholder business fields. Generated teaching images are excluded from the evaluation.

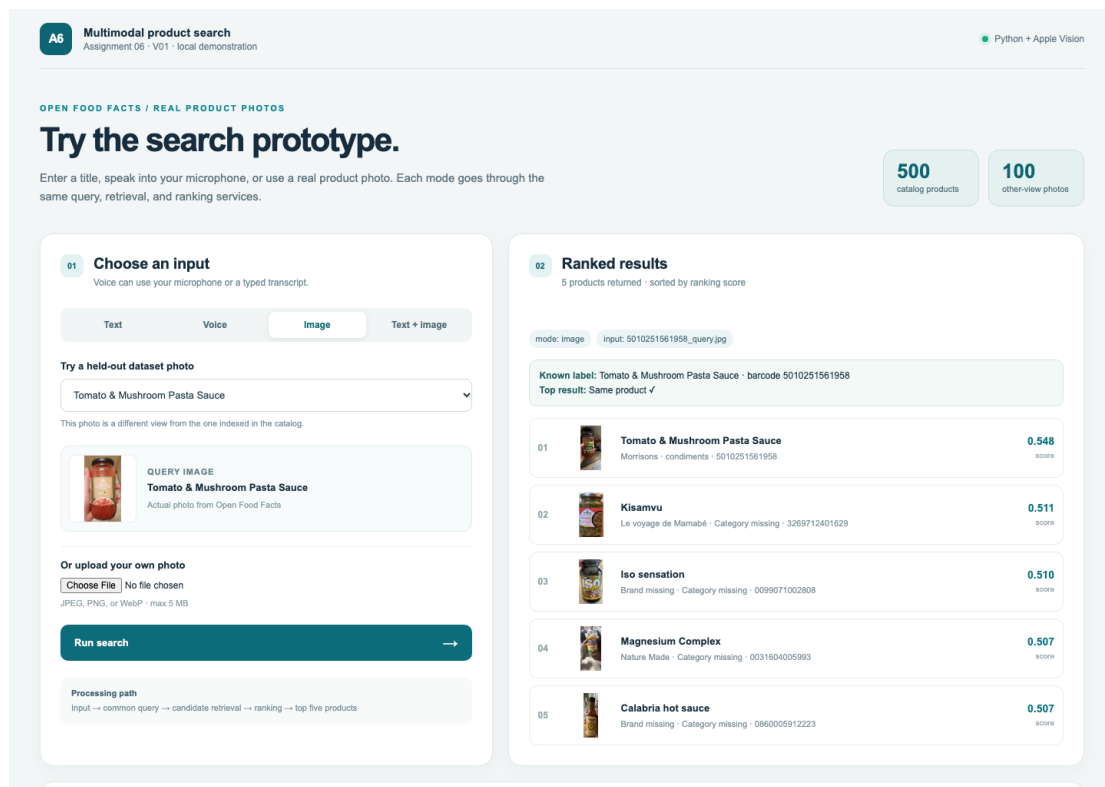


Figure 7: Local Python web demonstration searching with a real Open Food Facts query photo. The known barcode and ranked products are shown.

Table 7: Required search modes and one extension

Mode	Example input	First result
Text	find Nike shoes under 100 dollars	Nike Air Zoom Black Running Shoes
Voice	find black running shoes (console transcript)	Nike Air Zoom Black Running Shoes
Image	Alternate photo of Tomato & Mushroom Pasta Sauce	Tomato & Mushroom Pasta Sauce
Text + image	Tomato Mushroom Pasta Sauce and the photo	Tomato & Mushroom Pasta Sauce

The browser runs from `web_demo.py` at `127.0.0.1:8765`; it accepts a provided dataset photo or an uploaded photo. Its image result is a genuine Apple Vision result on the 500-product gallery. The first result is correct for the shown sauce query, while the full 100-photo evaluation below includes failures.

7 Evaluation

What is measured. Each test asks whether the application’s first result is correct. For exact-product image search, the correct answer has the same barcode as the query photo. We also count whether it appears anywhere in the first five results.

Source and test setup. Product records come from the official Open Food Facts random-modulo-1000 sample; photos come from its image store. A fixed selection provides 500 catalog products and 100 separate query photos. The catalog contains a product’s front photo, while its different query photo is kept out of the index. Thus, matching requires recognizing the same product across two views. The selection seed, source hash, and image attribution are supplied with the dataset.

Method. The test calls the submitted Python application’s SearchUI, SearchService, and Apple Vision index. The category check asks whether the first result has the same broad Open Food Facts category; 48 of the 100 queries lack a usable label and are excluded from that check. For text, the query is the catalog’s own product title, which is easier than a customer’s free-form request. Voice checks use known transcripts and assess retrieval after transcription; live microphone recognition is demonstrated but not scored.

Table 8: Results from the submitted application

Check	Correct first result	Interpretation
Teaching text and voice queries	16/16; plus 3/3 no-match checks	Basic query handling works on the small teaching catalog.
Same product, different real photo	22/100 (22%); 26/100 in top five	The exact item is often missed across different package views.
Same broad category	16/52 (31%)	Only 52 queries have a usable category label.
Exact product title	83/100 (83%)	This tests catalog-title lookup, not natural-language shopping.
Order access	4/4	Includes rejecting another customer’s order.

Examples and interpretation. The alternate photo of *Tomato & Mushroom Pasta Sauce* retrieves its matching front photo first. The *Quick cook medium noodles* query shows the back of the package; its matching front photo is outside the top five. Both image pairs are shown earlier in this report. The teaching check covers text and voice only; all image results come from real photos. Per-query results are saved in `demo/openfoodfacts_evaluation.json`.

8 Discussion and conclusion

The prototype meets the required tasks: it defines the customer requirements, models all required use cases and the three layers in Visual Paradigm, illustrates voice search, and implements text, voice-transcript, image, and ranked results in Python. The browser adds live microphone transcription. It also demonstrates text-image fusion, price filtering, and order ownership checks.

The main limitation is exact-product image retrieval: the Apple Vision method finds the correct product first for 22 of 100 alternate-view photos. It also requires macOS and Swift. The teaching catalog is small; the real-photo sample is grocery-focused and much smaller than a real store. Text evaluation uses exact product titles. Microphone recognition depends on browser support, permission, and the browser's recognition service; its accuracy is not measured here. The supplied customer ID is not an authentication mechanism.

A practical next step is to test product-specific image representations or package-text recognition on the same held-out photos. The current service boundaries allow that change within the image index without moving search logic into the user interface.

Reproducibility. The GitHub submission contains the report, editable .vpp project, Python source, browser interface, teaching records, README, thirteen automated tests, and saved evaluation results. The prepared Open Food Facts photos and metadata are provided as a separate ZIP on Google Drive. The README provides a tested gdown command, checksum, and extraction steps. The UML project can be rebuilt with `tools/build_vp.py`; the report uses the supplied Assignment 06 handout as its specification.

Submission links. GitHub source and report: github.com/scalliontor/A6_V01_VuHungAnh. Prepared dataset: drive.google.com/drive/folders/1Ta9vMurnYoJzvNcP23Iflulr-EVWIX58.

Dataset attribution. Product records are from Open Food Facts contributors. The database is under ODbL 1.0 and the Database Contents License; product images are under CC BY-SA 3.0. Selection and image resizing are documented in `data/openfoodfacts/DATASET.md` inside the Drive ZIP. This coursework is not endorsed by Open Food Facts. See the official dataset sample documentation and license guidance.